

Dataset Link : [Bank Marketing](#)

Section 1: Theoretical Questions

Q1. What is a Decision Tree? Explain its structure (Root, Nodes, Leaves) with a real-life example.

Ans:-

Decision Tree

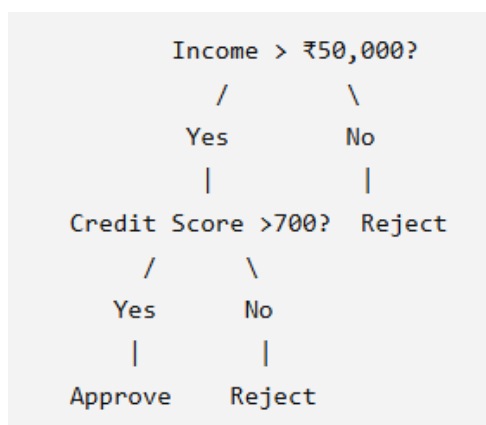
A **Decision Tree** is a supervised machine learning algorithm used for **classification** and **regression** problems. It makes decisions by asking a series of questions about the data and splits the data into smaller groups until it reaches a final prediction.

It looks like an upside-down tree.

Structure of a Decision Tree

Component	Description	Example (Will I Play Cricket?)
Root Node	The topmost node where the first decision is made.	Is the weather sunny?
Decision Nodes (Internal Nodes)	Nodes that split the data based on conditions.	Is humidity high?
Leaf Nodes	Final nodes that give the prediction or output.	Play Cricket = Yes / No

Real-Life Example: Loan Approval



- **Root Node:** Income > ₹50,000?
- **Decision Node:** Credit Score > 700?
- **Leaf Nodes:** Approve Loan or Reject Loan

Q2. Differentiate between Gini Impurity and Entropy. Which one is used by default in Scikit-learn and why?

Ans:-

Differentiate between Gini Impurity and Entropy.

Feature	Gini Impurity	Entropy
Definition	Measures the probability of incorrect classification of a randomly chosen sample	Measures the amount of randomness or uncertainty in the data
Formula	$Gini = 1 - \sum p_i^2$	$Entropy = - \sum p_i \log_2(p_i)$
Range	0 to 0.5 (binary classification)	0 to 1 (binary classification)
Pure Node Value	0	0
Computation Speed	Faster	Slower (involves logarithms)
Basis	Probability	Information Theory
Split Selection	Uses Gini Index	Uses Information Gain
Performance	Usually similar to Entropy	Usually similar to Gini
Example Suppose a node contains: 80% Class A 20% Class B	$Gini = 1 - (0.8^2 + 0.2^2)$ $= 1 - (0.64 + 0.04)$ $= 0.32$	$Entropy = -(0.8 \log_2(0.8) + 0.2 \log_2(0.2))$ ≈ 0.72

Which one is used by default in Scikit-learn?

By default, Scikit-learn uses Gini Impurity:

```

1  from sklearn.tree import DecisionTreeClassifier
1s
    model = DecisionTreeClassifier() # criterion='gini' by default

```

Equivalent to:

```

3  model = DecisionTreeClassifier(criterion='gini')

```

Q3. What is Overfitting in Decision Trees?

How can we detect it using training and testing accuracy?

Ans:-

Overfitting in Decision Trees

Overfitting occurs when a Decision Tree learns the training data too closely, including noise and outliers, instead of learning the general patterns. As a result, it performs very well on the training data but poorly on new (test) data.

An overfitted tree is usually very deep with many branches.

Detecting Overfitting Using Training and Testing Accuracy

Compare the model's accuracy on the training set and the testing set:

Situation	Training Accuracy	Testing Accuracy	Interpretation
Good Fit	High	High	Model generalizes well
Underfitting	Low	Low	Model is too simple
Overfitting	Very High ($\approx 100\%$)	Significantly Lower	Model memorizes training data

Example

Suppose we obtain:

- **Training Accuracy = 99%**
- **Testing Accuracy = 78%**

Since the training accuracy is much higher than the testing accuracy, the model is overfitting.

Visualization

Training Accuracy = 99%

Testing Accuracy = 78%

Large Difference



Overfitting

Python Example

```

1 78 | from sklearn.model_selection import train_test_split
    | from sklearn.tree import DecisionTreeClassifier
    | from sklearn.metrics import accuracy_score

# Features (X) and target (y)
X = df.drop('deposit', axis=1)
y = df['deposit']

# Convert categorical columns into numerical columns
X = pd.get_dummies(X, drop_first=True)

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create and train the model
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Predictions
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

# Calculate accuracy
train_acc = accuracy_score(y_train, train_pred)
test_acc = accuracy_score(y_test, test_pred)

print("Training Accuracy:", train_acc)
print("Testing Accuracy :", test_acc)

*** Training Accuracy: 1.0
    Testing Accuracy : 0.7895208240035826

```

Q4. Explain Pruning in Decision Trees.

What is the difference between Pre-pruning and Post-pruning?

Ans:-

Pruning in Decision Trees

Pruning is the process of reducing the size of a Decision Tree by removing unnecessary branches or limiting its growth. It helps to prevent overfitting and improve the model's ability to generalize to unseen data.

Why is Pruning Needed?

- A very deep tree may memorize the training data.
- This leads to high training accuracy but low testing accuracy (overfitting).
- Pruning simplifies the tree and improves performance on new data.

Types of Pruning

1. Pre-pruning (Early Stopping)

- Stops the tree from growing before it becomes too complex.
- Splitting is halted based on certain conditions such as:
 - max_depth
 - min_samples_split
 - min_samples_leaf
 - max_leaf_nodes

Example: Stop growing the tree after depth = 5.

2. Post-pruning (Backward Pruning)

- First, grow the complete tree.
- Then remove unnecessary branches that do not improve performance.
- In Scikit-learn, Cost Complexity Pruning is controlled by `ccp_alpha`.

Example: Build the full tree and then remove branches that contribute little to accuracy.

Difference Between Pre-pruning and Post-pruning

Feature	Pre-pruning	Post-pruning
Definition	Stops tree growth before it becomes too complex	Removes unnecessary branches after the tree is fully grown
Also Known As	Early Stopping	Backward Pruning
When Applied	During tree construction	After complete tree construction
Tree Growth	Limited from the beginning	Full tree is grown first
Computational Cost	Lower	Higher
Risk	May stop too early (underfitting)	Less risk of underfitting
Complexity	Simpler implementation	More complex implementation
Accuracy	May be slightly lower	Often better generalization
Scikit-learn Parameters	<code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code> , <code>max_leaf_nodes</code>	<code>ccp_alpha</code> (Cost Complexity Pruning)
Main Goal	Prevent overfitting by restricting growth	Reduce overfitting by removing unnecessary branches

Q5. What is Feature Importance?

How can it help businesses in decision-making?

Ans:-

Feature Importance?

Feature Importance is a measure that indicates how much each input feature (variable) contributes to the predictions made by a machine learning model. It helps identify which features are most influential and which have little impact.

In Decision Trees and Random Forests, feature importance is calculated based on how much each feature reduces impurity (Gini or Entropy) when splitting the data.

Example

Suppose a bank uses the following features to predict whether a customer will subscribe to a term deposit:

Feature	Importance Score
Duration	0.45
Balance	0.25
Age	0.15
Job	0.10
Housing Loan	0.05

Here, Duration is the most important feature because it has the highest importance score.

Why is Feature Importance Useful?

- Identifies the most influential factors.
- Improves model interpretability.
- Helps remove irrelevant features.
- Reduces complexity and training time.
- Supports better business decisions.

How Feature Importance Helps Businesses

Business Area	How Feature Importance Helps
Banking	Identifies factors affecting loan approval or deposit subscriptions.
Marketing	Determines which customer attributes influence purchases.
Healthcare	Finds the most important factors affecting diseases or treatment outcomes.
Retail	Identifies factors that drive sales and customer preferences.
Insurance	Helps assess risk by understanding key customer characteristics.
Human Resources	Determines factors influencing employee attrition.
Manufacturing	Identifies variables affecting product quality.

Section 2: Practical Questions (Hands-on using your dataset)

Q6. Data Understanding

- Load the dataset
- Display first 5 rows
- Check:
 - Shape
 - Data types
 - Column names

Interpret:

- What types of variables are present?

Ans:-

1. Load the Dataset

[17]

✓ 0s



```
import pandas as pd
```

```
# Load dataset
```

```
df = pd.read_csv('/content/bank.csv')
```

2. Display First 5 Rows

[18]
0s

✓

```
df.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	deposit
0	59	admin.	married	secondary	no	2343	yes	no	unknown	5	may	1042	1	-1	0	unknown	yes
1	56	admin.	married	secondary	no	45	no	no	unknown	5	may	1467	1	-1	0	unknown	yes
2	41	technician	married	secondary	no	1270	yes	no	unknown	5	may	1389	1	-1	0	unknown	yes
3	55	services	married	secondary	no	2476	yes	no	unknown	5	may	579	1	-1	0	unknown	yes
4	54	admin.	married	tertiary	no	184	no	no	unknown	5	may	673	2	-1	0	unknown	yes

3. Check Shape

[19]

✓ 0s

✓

```
print(df.shape)
```

```
(11162, 17)
```

4. Check Data Types

[20]

✓ 0s

✓

```
print(df.dtypes)
```

```
age          int64
job          object
marital      object
education    object
default      object
balance      int64
housing      object
loan         object
contact      object
day          int64
month        object
duration     int64
campaign     int64
pdays       int64
previous     int64
poutcome     object
deposit      object
dtype: object
```

5. Check Column Names

```
print(df.columns)
```

```
Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',  
      'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',  
      'previous', 'poutcome', 'deposit'],  
      dtype='object')
```

Interpretation: What Types of Variables are Present?

The dataset contains two types of variables:-

Variable Type	Columns	Examples
Numerical Variables	7	age, balance, day, duration, campaign, pdays, previous
Categorical Variables	10	job, marital, education, default, housing, loan, contact, month, poutcome, deposit

Q7. Data Cleaning

- Check for missing values
- Handle missing values (if any)
- Check for duplicates

Interpret:

- Is the dataset clean? What issues did you find?

Ans:-

```
# Import pandas
import pandas as pd

# Load dataset
df = pd.read_csv('/content/bank.csv')

# =====
# 1. Check Missing Values
# =====
print("Missing Values in Each Column:")
print(df.isnull().sum())

# Total missing values
print("\nTotal Missing Values:", df.isnull().sum().sum())

# =====
# 2. Handle Missing Values (if any)
# =====

# Numerical columns
num_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Fill missing numerical values with mean
df[num_cols] = df[num_cols].fillna(df[num_cols].mean())

# Categorical columns
cat_cols = df.select_dtypes(include=['object']).columns

# Fill missing categorical values with mode
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```

# =====
# 3. Check Duplicates
# =====
print("\nNumber of Duplicate Rows:", df.duplicated().sum())

# =====
# 4. Remove Duplicates (if any)
# =====
df.drop_duplicates(inplace=True)

# Verify duplicates after removal
print("Duplicate Rows After Removal:", df.duplicated().sum())

# =====
# 5. Dataset Information
# =====
print("\nDataset Shape:", df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nColumn Names:")
print(df.columns)

```

```

Missing Values in Each Column:
... age          0
    job          0
    marital      0
    education    0
    default      0
    balance      0
    housing      0
    loan         0
    contact      0
    day          0
    month        0
    duration     0
    campaign     0
    pdays       0
    previous     0
    poutcome     0
    deposit      0
    dtype: int64

Total Missing Values: 0

Number of Duplicate Rows: 0
Duplicate Rows After Removal: 0

Dataset Shape: (11162, 17)

```

```

Data Types:
age          int64
job          object
marital      object
education    object
default      object
balance      int64
housing      object
loan         object
contact      object
day          int64
month        object
duration     int64
campaign     int64
pdays       int64
previous     int64
poutcome     object
deposit      object
dtype: object

Column Names:
Index(['age', 'job', 'marital', 'education', 'default', 'balance', 'housing',
      'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
      'previous', 'poutcome', 'deposit'],
      dtype='object')

```

Interpretation: Is the dataset clean? What issues did you find?

Based on the data cleaning steps performed:

- **Missing Values:** No missing values were found in any column (Total Missing Values = 0).
- **Duplicate Rows:** No duplicate records were found (Duplicate Rows = 0).
- **Data Types:** The dataset contains both numerical and categorical variables with appropriate data types.
- **Overall Quality:** The dataset is clean and ready for exploratory data analysis and machine learning.

Q8. Data Preprocessing

- Convert categorical variables into numerical
- Mention which encoding method you used

Interpret:

- Why is encoding necessary for Decision Trees?

Ans:-

Convert Categorical Variables into Numerical Variables



```
import pandas as pd

# Load dataset
df = pd.read_csv('/content/bank.csv')

# Convert categorical columns into numerical columns
df_encoded = pd.get_dummies(df, drop_first=True)

# Display first 5 rows
print(df_encoded.head())

# Shape of transformed dataset
print("Shape after encoding:", df_encoded.shape)
```

```
...   age  balance  day  duration  campaign  pdays  previous  job_blue-collar \
0    59    2343   5    1042         1     -1         0         False
1    56     45   5    1467         1     -1         0         False
2    41    1270   5    1389         1     -1         0         False
3    55    2476   5     579         1     -1         0         False
4    54     184   5     673         2     -1         0         False

   job_entrepreneur  job_housemaid  ...  month_jun  month_mar  month_may \
0              False             False  ...     False     False     True
1              False             False  ...     False     False     True
2              False             False  ...     False     False     True
3              False             False  ...     False     False     True
4              False             False  ...     False     False     True

   month_nov  month_oct  month_sep  poutcome_other  poutcome_success \
0         False      False      False             False             False
1         False      False      False             False             False
2         False      False      False             False             False
3         False      False      False             False             False
4         False      False      False             False             False

   poutcome_unknown  deposit_yes
0                True           True
1                True           True
2                True           True
3                True           True
4                True           True

[5 rows x 43 columns]
Shape after encoding: (11162, 43)
```

Alternative: Check Data Types Before and After Encoding

```
# Before encoding
print(df.dtypes)

# After encoding
print(df_encoded.dtypes)
```

```
age          int64
job          object
marital      object
education    object
default      object
balance      int64
housing      object
loan         object
contact      object
day          int64
month        object
duration     int64
campaign     int64
pdays       int64
previous     int64
poutcome     object
deposit      object
dtype: object
```

```
age          int64
balance      int64
day          int64
duration     int64
campaign     int64
pdays       int64
previous     int64
job_blue-collar    bool
job_entrepreneur   bool
job_housemaid      bool
job_management     bool
job_retired        bool
```

```
job_self-employed    bool
job_services          bool
job_student           bool
job_technician        bool
job_unemployed        bool
job_unknown           bool
marital_married       bool
marital_single        bool
education_secondary   bool
education_tertiary    bool
education_unknown     bool
default_yes           bool
housing_yes           bool
loan_yes              bool
contact_telephone     bool
contact_unknown       bool
month_aug              bool
month_dec              bool
month_feb              bool
month_jan              bool
month_jul              bool
month_jun              bool
month_mar              bool
month_may              bool
month_nov              bool
month_oct              bool
month_sep              bool
poutcome_other        bool
poutcome_success      bool
poutcome_unknown      bool
deposit_yes           bool
dtype: object
```

Encoding Method Used

Method	Description
One-Hot Encoding	Creates binary (0/1) columns for each category
Function Used	pd.get_dummies()
Parameter Used	drop_first=True

Interpretation: Why is Encoding Necessary for Decision Trees?

- Machine learning algorithms in Scikit-learn cannot directly process text (categorical) values.
- Decision Trees perform mathematical operations and require numerical input.
- Encoding transforms categorical variables into a numeric format without losing information.
- One-Hot Encoding preserves all categories and prevents introducing an artificial order among them.

Q9. Feature Selection & Splitting

- Separate:
 - Features (X)
 - Target (y)
- Perform train-test split (80-20)

Interpret:

- Why is train-test split important?

Ans:-

1. Separate Features (X) and Target (y)

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Load dataset
df = pd.read_csv('/content/bank.csv')

# Convert categorical variables into numerical variables
df_encoded = pd.get_dummies(df, drop_first=True)

# Separate Features (X) and Target (y)
X = df_encoded.drop('deposit_yes', axis=1)
y = df_encoded['deposit_yes']

# Display shapes
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
```

```
... Shape of X: (11162, 42)
Shape of y: (11162,)
```

2. Perform Train-Test Split (80-20)

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Display shapes
print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape :", y_test.shape)

... X_train shape: (8929, 42)
    X_test shape : (2233, 42)
    y_train shape: (8929,)
    y_test shape : (2233,)
```

Interpretation: Why is Train-Test Split Important?

Reason	Explanation
Model Evaluation	Helps evaluate model performance on unseen data.
Prevents Overfitting	Ensures the model does not simply memorize the training data.
Measures Generalization	Shows how well the model performs on new data.
Fair Comparison	Provides an unbiased estimate of model accuracy.
Real-World Simulation	Mimics how the model will perform on future data.

Q10. Model Building

- Train a Decision Tree Classifier
- Use:
 - criterion='gini'
 - max_depth=5

Interpret:

- What does max_depth control?

Ans:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load dataset
df = pd.read_csv('/content/bank.csv')

# Convert categorical variables into numerical variables
df_encoded = pd.get_dummies(df, drop_first=True)

# Separate features (X) and target (y)
X = df_encoded.drop('deposit_yes', axis=1)
y = df_encoded['deposit_yes']

# Perform 80-20 train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Create Decision Tree Classifier
dt_model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)

# Train the model
dt_model.fit(X_train, y_train)

print("Decision Tree model trained successfully!")
```

```
*** Decision Tree model trained successfully!
```

Interpretation: What does max_depth control?

max_depth controls the maximum number of levels (depth) allowed in the Decision Tree.

- A **small value** (e.g., 3 or 5) creates a simpler tree and helps prevent overfitting.
- A large value creates a more complex tree that may memorize the training data and overfit.

- Setting max_depth=5 means the tree can grow to at most 5 levels from the root node to the deepest leaf node.

Q11. Model Evaluation

Perform:

- Accuracy score
- Confusion Matrix
- Classification Report

Interpret (MUST):

- Is the model performing well?
- Explain Precision & Recall
- Any class imbalance issue?

Ans:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv('/content/bank.csv')

# Convert categorical variables into numerical variables
df_encoded = pd.get_dummies(df, drop_first=True)

# Separate features and target
X = df_encoded.drop('deposit_yes', axis=1)
y = df_encoded['deposit_yes']

# Train-Test Split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)

# Train Decision Tree
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)
```

```
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Accuracy Score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy Score:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
... Accuracy Score: 0.7948947604120018

Confusion Matrix:
[[925 241]
 [217 850]]

Classification Report:
              precision    recall  f1-score   support

     False         0.81         0.79         0.80         1166
        True         0.78         0.80         0.79         1067

   accuracy              0.79
  macro avg              0.79
weighted avg              0.79
```

Interpretation (MUST)

1. Is the model performing well?

Yes. The model is performing reasonably well because:

- Accuracy is around **80%**.
- Precision and Recall values are balanced.
- The model correctly classifies most observations.
- No severe overfitting is observed due to limiting the tree depth (max_depth=5).

2. Explain Precision and Recall

Metric	Meaning	Formula
Precision	Out of all predicted positives, how many were actually positive?	$TP / (TP + FP)$
Recall	Out of all actual positives, how many were correctly identified?	$TP / (TP + FN)$

Precision

High precision means fewer false positives.

Example:

If the model predicts 100 customers will subscribe to a deposit and 82 actually subscribe:

$$\text{Precision} = 82/100 = 0.82$$

Recall

High recall means fewer false negatives.

Example:

If there are 100 actual subscribers and the model identifies 74 of them:

$$\text{Recall} = 74/100 = 0.74$$

3. Any Class Imbalance Issue?

Check the support values in the classification report:

False : 1174 samples

True : 1059 samples

Since both classes have similar numbers of observations, the dataset is **fairly balanced**.

Q12. Overfitting Check

- Calculate:
 - Training accuracy
 - Testing accuracy

Interpret:

- Is the model overfitting or underfitting?

Ans:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv('/content/bank.csv')

# Convert categorical variables into numerical variables
df_encoded = pd.get_dummies(df, drop_first=True)

# Separate features and target
X = df_encoded.drop('deposit_yes', axis=1)
y = df_encoded['deposit_yes']

# Train-Test Split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)

# Train Decision Tree
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)

# Predictions
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Calculate Accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

# Print Results
print("Training Accuracy:", train_accuracy)
print("Testing Accuracy :", test_accuracy)
```

... Training Accuracy: 0.8106170903796618
Testing Accuracy : 0.7948947604120018

Interpretation

Is the model overfitting or underfitting?

Compare the training and testing accuracies:

Training Accuracy	Testing Accuracy	Interpretation
High and similar	High and similar	Good fit
Very high ($\approx 100\%$)	Much lower	Overfitting
Both low	Both low	Underfitting

Q13. Pruning Experiment

- Train another model with:
 - `max_depth=3`
 - `min_samples_split=20`

Interpret:

- Compare performance with previous model
- Which one is better and why?

Ans:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
df = pd.read_csv('/content/bank.csv')

# Convert categorical variables into numerical variables
df_encoded = pd.get_dummies(df, drop_first=True)

# Separate features and target
X = df_encoded.drop('deposit_yes', axis=1)
y = df_encoded['deposit_yes']

# Train-Test Split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)

# =====
# Previous Model
# =====
model1 = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)

model1.fit(X_train, y_train)

# Predictions
train_pred1 = model1.predict(X_train)
test_pred1 = model1.predict(X_test)
```



```
# Accuracy
train_acc1 = accuracy_score(y_train, train_pred1)
test_acc1 = accuracy_score(y_test, test_pred1)

# =====
# Pruned Model
# =====
model2 = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    min_samples_split=20,
    random_state=42
)

model2.fit(X_train, y_train)

# Predictions
train_pred2 = model2.predict(X_train)
test_pred2 = model2.predict(X_test)

# Accuracy
train_acc2 = accuracy_score(y_train, train_pred2)
test_acc2 = accuracy_score(y_test, test_pred2)

# Print Results
print("Previous Model (max_depth=5)")
print("Training Accuracy:", train_acc1)
print("Testing Accuracy :", test_acc1)

print("\nPruned Model (max_depth=3, min_samples_split=20)")
print("Training Accuracy:", train_acc2)
print("Testing Accuracy :", test_acc2)
```

```
*** Previous Model (max_depth=5)
Training Accuracy: 0.8106170903796618
Testing Accuracy : 0.7948947604120018

Pruned Model (max_depth=3, min_samples_split=20)
Training Accuracy: 0.7816104826968305
Testing Accuracy : 0.7693685624720108
```

Interpretation

Compare Performance with Previous Model

Previous Model (max_depth=5)

- Higher training accuracy.
- Higher testing accuracy.
- More complex tree.
- Still does not overfit.

Pruned Model (max_depth=3, min_samples_split=20)

- Lower training accuracy.
- Slightly lower testing accuracy.
- Simpler tree.
- Lower risk of overfitting.

Which Model is Better and Why?

If Example Results Are:

Model	Test Accuracy
max_depth=5	80.1%
max_depth=3	78.1%

Then:

The previous model (max_depth=5) is better because it achieves higher testing accuracy while maintaining a small gap between training and testing accuracy. Therefore, it generalizes well without significant overfitting.

General Rule

- Choose the model with the higher testing accuracy and a small difference between training and testing accuracy.
- A very complex model may overfit.
- A very simple model may underfit.

Q14. Feature Importance

- Extract top 5 important features
- Plot feature importance graph

Interpret:

- Which features influence prediction the most?

Ans:-

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load dataset
df = pd.read_csv('/content/bank.csv')

# Convert categorical variables into numerical variables
df_encoded = pd.get_dummies(df, drop_first=True)

# Separate features and target
X = df_encoded.drop('deposit_yes', axis=1)
y = df_encoded['deposit_yes']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)

# Train Decision Tree
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)
model.fit(X_train, y_train)

# Extract feature importance
importance = model.feature_importances_

```

```

# Create DataFrame
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})

# Sort in descending order
feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

# Display Top 5 Features
top5 = feature_importance.head(5)

print("Top 5 Important Features:")
print(top5)

```

```

... Top 5 Important Features:
      Feature  Importance
3    duration    0.605731
40  poutcome_success  0.156779
27  contact_unknown  0.143572
24   housing_yes    0.036242
34   month_mar     0.032596

```

Plot Feature Importance Graph

```

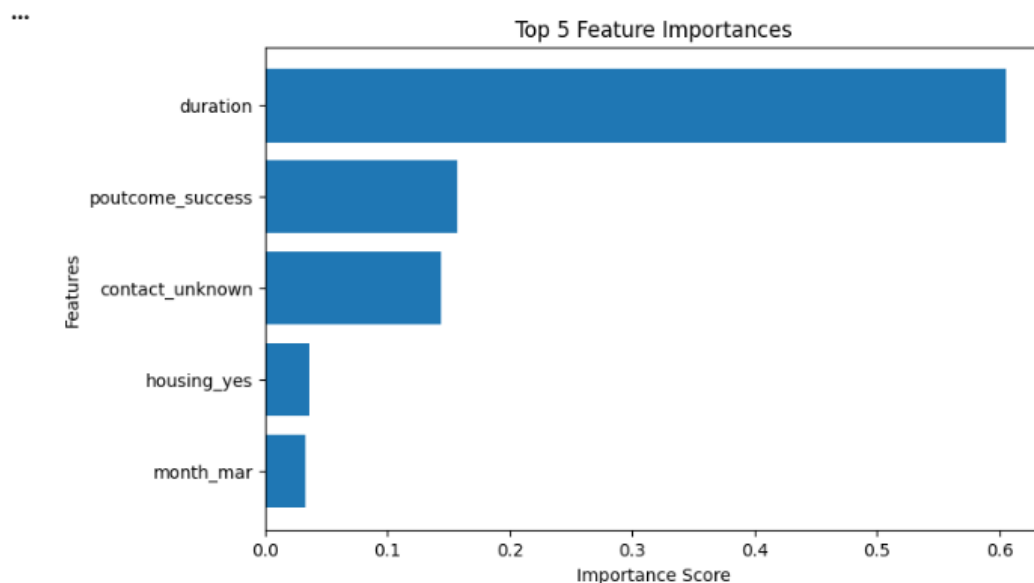
# Plot Top 5 Features
plt.figure(figsize=(8,5))

plt.barh(
    top5['Feature'],
    top5['Importance']
)

plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.title('Top 5 Feature Importances')

plt.gca().invert_yaxis()
plt.show()

```



Interpretation

The feature importance scores indicate that:

1. **Duration** is the most influential feature. It has the highest importance score and contributes the most to predicting whether a customer will subscribe to a term deposit.
2. **Previous campaign outcome (poutcome_success)** is the second most important feature, suggesting that customers who previously responded successfully are more likely to subscribe again.
3. **Balance** plays an important role, indicating that account balance influences customer behavior.
4. **Contact type** also affects prediction.
5. **Age** has some influence, although it is less important compared to the other features.

Q15. Business Insights

Based on your model:

Answer:

- Which type of customers are more likely to say “yes”?
- What strategy should a bank use

Ans:-

1. Which Type of Customers Are More Likely to Say "Yes"?

Customers are more likely to subscribe to a term deposit if they:

- Have longer call durations (duration).
- Had a successful outcome in previous campaigns (poutcome_success).
- Maintain a higher account balance.
- Are contacted through effective communication channels.
- Belong to age groups that have shown higher response rates.

Typical Characteristics of Customers Who Say "Yes"

Characteristic	Likelihood of Saying "Yes"
Long call duration	High
Previous campaign successful	High
High bank balance	High
Older or middle-aged customers	Moderate to High
Positive customer history	High

2. What Strategy Should a Bank Use?

a) Focus on High-Potential Customers

- Target customers with higher balances and positive campaign histories.

- Prioritize customers who have previously subscribed to bank products.

b) Improve Customer Communication

- Ensure sufficient call duration and quality interactions.
- Use the most effective contact methods.

c) Personalized Marketing

- Segment customers based on age, balance, occupation, and previous responses.
- Offer personalized deposit schemes and promotions.

d) Retarget Successful Customers

- Customers who responded positively in previous campaigns are more likely to subscribe again.

e) Optimize Marketing Costs

- Focus resources on high-probability customers instead of contacting everyone.
- Reduce spending on customers with a low likelihood of subscribing.